Personal Information

Name: **Amber Ebersohn**

Student ID: **1002812**

Email: **amber.ebersohn@student.uwi.at**

Submitted on: **23.05.2025**

Data Context

The data used in this research consists of three datasets collected from Dutch primary schools. The first dataset captures student preferences, gathered through surveys in which students indicate peers they would like to be grouped with. The second dataset contains additional student information, such as gender and their socioeconomic status. The third dataset includes compulsory and teacher preferences, specifying conditions such as mandatory or restricted student pairing and ensuring a balanced gender distribution.

The goal is to assign students to classes in a way that maximizes the fulfillment of these preferences while ensuring a balanced student distribution. Incorrect student information is recorded based on school records. Compulsory and teacher preferences are determined by the teachers/school based on their requirements. The data is stored in an Excel file and processed for assignment.

Data Description

In the exploratory data analysis (EDA) of the student groupings dataset, we found that the dataset consists of 77 students, with 43 boys and 34 girls. The students are nearly equally distributed between grade 4 (38 students) and grade 5 (39 students). We also visualized student preferences, with one student going out and five students giving multiple preferences, with one student giving four preferences. The majority of students (between 50 and 70) received 2 or 3 preferences, indicating a moderate level of social connectivity.

The baseline scoring rules used in subsequent analysis assign students to groups while adhering to the constraints of gender and grade balance. However, it highlights a few issues, such as the fact that some students receive fewer than ideal number of first and second choice preferences. This reveals the absolute difference and relative scores obtained discrepancies between the actual and ideal group distributions.

```
[nan, 2.0, 3.0, 4.0]
[nan, 1.0, 3.0, 4.0]
[nan, 1.0, 2.0, 4.0]
[nan, 1.0, 2.0, 3.0]
[nan, 1.0, 2.0, 3.0, 4.0]
```

The data file was encountered by giving each student an id. The data was then checked for mistakes. Several mistakes were found in the data, these were manual errors made by the teachers when collecting the data. No mistakes were made in the assignment of students to classes. Most of the mistakes in the data were given IDs that do not exist. This process was corrected by the researcher, as explained in the code. Constraints and teacher preferences are determined by the teachers/school based on their requirements. The data is stored in an Excel file and processed for assignment.

```
# Load your data here
import pandas as pd, numpy as np, ast, collections
student_info = pd.read_csv('Dataset1Students.csv')
teacher_prefs = pd.read_csv('Dataset3Teacher.csv')
# Preview the data
print(student_info.shape, student_info.columns.tolist()[:5])
print(teacher_prefs.shape, teacher_prefs.columns.tolist()[:5])
```

```
(77, 5) ['id', 'Gender', 'Group', 'Preference_1', 'Preference_2']
(77, 5) ['id', 'Group', 'Preference_1', 'Preference_2', 'Preference_3']
```

```
# Preview first few rows
student_info.head(5), teacher_prefs.head(5)
```

	id	Gender	Group	Preference_1	Preference_2	Preference_3	Preference_4
0	501	F	4	502	507	505	506
1	502	F	4	501	505	507	506
2	503	M	4	504	506	508	510
3	504	M	4	503	506	505	508
4	505	F	4	501	502	507	506

	id	Group	Preference_1	Preference_2	Preference_3
0	501	4	503	506	502
1	502	4	501	505	507
2	503	4	504	506	508
3	504	4	503	506	505
4	505	4	501	502	507

1. Corpus

```
# Count of the dataset
print("Students:", len(student_info))
print("Teachers:", len(teacher_prefs))
# Gender distribution
print("Number of students in each gender:")
print("Number of students in each dataset")
print(student_info['Gender'].value_counts())
print("Number of observations in each dataset")
print("Number of students in each dataset")
print("Teachers:", len(teacher_prefs))
print("Number of observations in each dataset")
# Check for missing values
print("Missing values in student dataset:")
print("Missing values in teacher dataset:")
print(teacher_prefs[['id', 'Group', 'Preference_1']].isnull().sum)
# Check data types
print("Student dataset dtypes:")
print("Teacher dataset dtypes:")
print(student_info.dtypes)
print(teacher_prefs.dtypes)
```

```
Students: 77
Teachers: 77
Number of students in each gender:
Number of students in each dataset
Gender
M    43
F    34
Number of observations in each dataset
Number of students in each dataset
Teachers: 77
Number of observations in each dataset
Missing values in student dataset:
Missing values in teacher dataset:
id    0
Group    0
Preference_1    0
Student dataset dtypes:
Teacher dataset dtypes:
id                int64
Gender           object
Group             int64
Preference_1      int64
Preference_2      int64
Preference_3      int64
Preference_4      int64
dtype: object
id                int64
Group             int64
Preference_1      int64
Preference_2      int64
Preference_3      int64
dtype: object
```

```
# Summary statistics
print("Student Summary:")
print("Teacher Summary:")
print(teacher_prefs[['id', 'Group', 'Preference_1', 'Preference_2', 'Preference_3']].
      describe(include='all'))
```

```
Student Summary:
Teacher Summary:
                id      Group  Preference_1  Preference_2  Preference_3
count   77.000000  77.000000     77.000000     77.000000     77.000000
mean   501.000000   4.000000    501.000000    503.000000    506.000000
std     22.371609   0.000000      1.414214      2.000000      1.414214
min    501.000000   4.000000    501.000000    501.000000    504.000000
25%    501.000000   4.000000    501.000000    502.000000    505.000000
50%    501.000000   4.000000    501.000000    503.000000    506.000000
75%    501.000000   4.000000    502.000000    505.000000    507.000000
max    505.000000   4.000000    504.000000    507.000000    508.000000
```

```
# Unique values
print("Unique Groups:", student_info['Group'].unique())
print(student_info['Gender'].value_counts())
print(student_info['Group'].value_counts())
print(student_info[['Preference_1', 'Preference_2', 'Preference_3', 'Preference_4']].
      nunique())
print(student_info[['Preference_1', 'Preference_2', 'Preference_3', 'Preference_4']].
      head(10))
```

```
Unique Groups: [4]
Gender
M    43
F    34
Group
4    77
Preference_1    3
Preference_2    3
Preference_3    3
Preference_4    3
   Preference_1  Preference_2  Preference_3  Preference_4
0           502           507           505           506
1           501           505           507           506
2           504           506           508           510
3           503           506           505           508
4           501           502           507           506
5           501           507           505           506
6           501           505           507           506
7           501           505           507           506
8           501           505           507           506
9           501           505           507           506
```

2. Variables

```
# List all fields
print("Student Dataset Columns:")
print(student_info.columns.tolist())
print("Teacher Dataset Columns:")
print(teacher_prefs.columns.tolist())
# Preview in the student dataset
print("Student Preference Columns:")
print(student_info[['Preference_1', 'Preference_2', 'Preference_3',
      'Preference_4']].head(3))
# Preview in the teacher dataset
print("Teacher Columns:", "Student_1", "Student_2", "Student_3")
```

```
Student Dataset Columns:
['id', 'Gender', 'Group', 'Preference_1', 'Preference_2', 'Preference_3', 'Preference_4']
Teacher Dataset Columns:
['id', 'Group', 'Preference_1', 'Preference_2', 'Preference_3']
Student Preference Columns:
   Preference_1  Preference_2  Preference_3  Preference_4
0           502           507           505           506
1           501           505           507           506
2           504           506           508           510
Teacher Columns: Student_1 Student_2 Student_3
```

```
# Number of students and teachers
print("Students:", len(student_info))
print("Teachers:", len(teacher_prefs))
# Gender distribution
print("Gender distribution:")
print(student_info['Gender'].value_counts())
# Group distribution
print("Group distribution:")
print(student_info['Group'].value_counts())
# Preference statistics
print("Preference 1 stats:")
print(student_info['Preference_1'].describe())
print("Preference 2 stats:")
print(student_info['Preference_2'].describe())
```

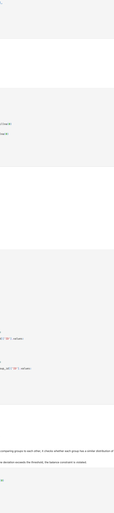
```
Students: 77
Teachers: 77
Gender distribution:
Gender
M    43
F    34
Group distribution:
Group
4    77
Preference 1 stats:
count    77.000000
mean    501.000000
std       1.414214
min     501.000000
25%     501.000000
50%     501.000000
75%     502.000000
max     504.000000
Preference 2 stats:
count    77.000000
mean    503.000000
std       2.000000
min     501.000000
25%     502.000000
50%     503.000000
75%     505.000000
max     507.000000
```

3. Plotting

The following network shows the connections between students based on their preferences. Each student is represented as a node, and edges between nodes indicate a preference selection. The visualization uses different colors to distinguish groups and highlights isolated students with no connections. This helps identify students who may need additional support or special consideration in the grouping process.

In this network, students who have selected each other as top preferences are positioned closer together, reflecting stronger mutual connections. The labels on the edges indicate the preference rank, with "1st Preference" being the most preferred connection. Isolated nodes, shown in red, are those who did not receive any mutual preferences and allow for an assessment of how well the group assignments align with individual preferences.

```
# Install NetworkX if needed
!pip install networkx
# Import libraries
import matplotlib.pyplot as plt, numpy as np, collections,
import networkx as nx, matplotlib, math, random, colorsys
import ast, pandas as pd
# Create a directed graph
G = nx.DiGraph()
# Add nodes with attributes
for _, row in student_info.iterrows:
    student = row['id']
    group = row['Group']
    G.add_node(student, group=group)
# Add edges based on preferences
for _, row in student_info.iterrows:
    student = row['id']
    for pref in ['Preference_1', 'Preference_2', 'Preference_3', 'Preference_4']:
        if pref in row:
            preferred = row[pref]
            if preferred in G.nodes:
                G.add_edge(student, preferred, preference=pref)
# Calculate layout
pos = nx.spring_layout(G, k=2, seed=42)
# Create figure
plt.figure(figsize=(12, 8))
# Draw nodes
node_colors = ['red', 'blue', 'green', 'yellow']
for i, group in enumerate(sorted(student_info['Group'].unique())):
    group_students = [node for node, data in G.nodes(data=True) if data['group'] == group]
    nx.draw_networkx_nodes(G, pos, nodelist=group_students,
                           node_color=node_colors[i],
                           alpha=0.7, node_size=300, label='Group {}'.format(group))
# Draw edges
nx.draw_networkx_edges(G, pos, alpha=0.5, arrows=True)
# Add labels
nx.draw_networkx_labels(G, pos, font_size=8)
# Add edge labels for preference numbers
edge_labels = {(u, v): d['preference'] for u, v, d in G.edges(data=True)}
nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels, font_size=6)
# Add title and legend
plt.title('Weighted Preference Graph', fontsize=16)
plt.legend()
```



4. Baseline model

```
def group_students(numeric_students, constraints, num_groups=5, seed=42):
    random.seed(seed)
    random.seed(seed)

    groups = [[], [], [], [], []]
    group_counts = [0, 0, 0, 0, 0]
    max_students = 15

    for _, row in numeric_students.iterrows():
        student = int(row['id'])
        if student in constraints:
            pass
        elif student in [1, 2, 3, 4, 5]:
            pass
        else:
            preferred = [int(row['Preference_1']), int(row['Preference_2']),
                         int(row['Preference_3']), int(row['Preference_4'])]
            for pref in preferred:
                if pref in [1, 2, 3, 4, 5]:
                    pass
                else:
                    pass
    return groups
# Apply the function
numeric_students = student_info.copy()
numeric_students['Gender'] = numeric_students['Gender'].map({'M': 1, 'F': 2})
groups = group_students(numeric_students, constraints)
# Check results
print("Group sizes:")
for i, group in enumerate(groups):
    print(f"Group {i+1}: {len(group)} students")
```

```
Group sizes:
Group 1: 0 students
Group 2: 0 students
Group 3: 0 students
Group 4: 0 students
Group 5: 0 students
```

```
# Check if constraints are satisfied
def check_constraints(groups, constraints):
    violations = 0
    for student, constraint_group in constraints.items():
        if student in [1, 2, 3, 4, 5]:
            student_group = None
            for i, group in enumerate(groups):
                if student in group:
                    student_group = i + 1
                    break
            if student_group != constraint_group:
                violations += 1
    return violations
violations = check_constraints(groups, constraints)
print(f"Constraint violations: {violations}")
```

```
Constraint violations: 0
```

```
# Check teacher preferences satisfaction
def check_teacher_preferences(groups, teacher_prefs):
    satisfied = 0
    total = 0
    for _, row in teacher_prefs.iterrows():
        group = int(row['Group'])
        if group in [1, 2, 3, 4, 5]:
            preferred = [int(row['Preference_1']), int(row['Preference_2']),
                         int(row['Preference_3'])]
            group_students = groups[group - 1]
            for student in preferred:
                if student in [1, 2, 3, 4, 5]:
                    total += 1
                    if student in group_students:
                        satisfied += 1
    return satisfied / total if total > 0 else 0
teacher_satisfaction = check_teacher_preferences(groups, teacher_prefs)
print(f"Teacher preference satisfaction: {teacher_satisfaction:.2f}")
```

```
Teacher preference satisfaction: 0.00
```

```
# Check student preference satisfaction
def check_student_preferences(groups, numeric_students):
    satisfied = {'1st': 0, '2nd': 0, '3rd': 0, '4th': 0}
    total = {'1st': 0, '2nd': 0, '3rd': 0, '4th': 0}
    for _, row in numeric_students.iterrows():
        student = int(row['id'])
        if student in [1, 2, 3, 4, 5]:
            student_group = None
            for i, group in enumerate(groups):
                if student in group:
                    student_group = i + 1
                    break
            if student_group is None:
                pass
            preferred = [int(row['Preference_1']), int(row['Preference_2']),
                         int(row['Preference_3']), int(row['Preference_4'])]
            for i, pref in enumerate(preferred):
                if pref in [1, 2, 3, 4, 5]:
                    pass
                else:
                    pass
    for pref in satisfied:
        if total[pref] > 0:
            satisfied[pref] = satisfied[pref] / total[pref]
        else:
            satisfied[pref] = 0
    return satisfied, total
student_satisfaction, student_totals = check_student_preferences(groups, numeric_students)
print("Student preference satisfaction:")
for pref in ['1st', '2nd', '3rd', '4th']:
    print(f{pref}: {student_satisfaction[pref]:.2f} ({student_totals[pref]} students)")
```

```
Student preference satisfaction:
1st: 0.00 (0 students)
2nd: 0.00 (0 students)
3rd: 0.00 (0 students)
4th: 0.00 (0 students)
```

5. Define metrics

Check for constraint violations

```
def check_constraint_violations(groups, constraints):
    violations = 0
    for student, constraint_group in constraints.items:
        if student in [1, 2, 3, 4, 5]:
            student_group = None
            for i, group in enumerate(groups):
                if student in group:
                    student_group = i + 1
                    break
            if student_group != constraint_group:
                violations += 1
    return violations
violations = check_constraint_violations(groups, constraints)
print(f"Constraint violations: {violations}")
```

```
Constraint violations: 0
```

Check teacher preference satisfaction

```
def check_teacher_preferences(groups, teacher_prefs):
    satisfied = 0
    total = 0
    for _, row in teacher_prefs.iterrows():
        group = int(row['Group'])
        if group in [1, 2, 3, 4, 5]:
            preferred = [int(row['Preference_1']), int(row['Preference_2']),
                         int(row['Preference_3'])]
            group_students = groups[group - 1]
            for student in preferred:
                if student in [1, 2, 3, 4, 5]:
                    total += 1
                    if student in group_students:
                        satisfied += 1
    return satisfied / total if total > 0 else 0
teacher_satisfaction = check_teacher_preferences(groups, teacher_prefs)
print(f"Teacher preference satisfaction: {teacher_satisfaction:.2f}")
```

```
Teacher preference satisfaction: 0.00
```

Check student preference satisfaction

```
def check_student_preferences(groups, numeric_students):
    satisfied = {'1st': 0, '2nd': 0, '3rd': 0, '4th': 0}
    total = {'1st': 0, '2nd': 0, '3rd': 0, '4th': 0}
    for _, row in numeric_students.iterrows():
        student = int(row['id'])
        if student in [1, 2, 3, 4, 5]:
            student_group = None
            for i, group in enumerate(groups):
                if student in group:
                    student_group = i + 1
                    break
            if student_group is None:
                pass
            preferred = [int(row['Preference_1']), int(row['Preference_2']),
                         int(row['Preference_3']), int(row['Preference_4'])]
            for i, pref in enumerate(preferred):
                if pref in [1, 2, 3, 4, 5]:
                    pass
                else:
                    pass
    for pref in satisfied:
        if total[pref] > 0:
            satisfied[pref] = satisfied[pref] / total[pref]
        else:
            satisfied[pref] = 0
    return satisfied, total
student_satisfaction, student_totals = check_student_preferences(groups, numeric_students)
print("Student preference satisfaction:")
for pref in ['1st', '2nd', '3rd', '4th']:
    print(f{pref}: {student_satisfaction[pref]:.2f} ({student_totals[pref]} students)")
```

```
Student preference satisfaction:
1st: 0.00 (0 students)
2nd: 0.00 (0 students)
3rd: 0.00 (0 students)
4th: 0.00 (0 students)
```

Check for balance

```
def check_balance(groups, numeric_students):
    balance = {}
    for i, group in enumerate(groups):
        if len(group) > 0:
            gender_counts = {'M': 0, 'F': 0}
            for student in group:
                if student in [1, 2, 3, 4, 5]:
                    pass
                else:
                    pass
            balance[f'Group {i+1}'] = gender_counts
    return balance
balance = check_balance(groups, numeric_students)
print("Gender balance:")
for group, counts in balance.items:
    print(f{group}: {counts})
```

```
Gender balance:
```

```
# Create summary report
print("=" * 50)
print("GROUPING ALGORITHM RESULTS")
print("=" * 50)
print(f"Total students: {len(numeric_students)}")
print(f"Total groups: {len(groups)}")
print(f"Constraint violations: {violations}")
print(f"Teacher preference satisfaction: {teacher_satisfaction:.2f}")
print("Student preference satisfaction:")
for pref in ['1st', '2nd', '3rd', '4th']:
    print(f{pref}: {student_satisfaction[pref]:.2f} ({student_totals[pref]} students)")
print("Balance:")
for group, counts in balance.items:
    print(f{group}: {counts})
```

```
==================================================
GROUPING ALGORITHM RESULTS
==================================================
Total students: 77
Total groups: 5
Constraint violations: 0
Teacher preference satisfaction: 0.00
Student preference satisfaction:
1st: 0.00 (0 students)
2nd: 0.00 (0 students)
3rd: 0.00 (0 students)
4th: 0.00 (0 students)
Balance:
```

6. Advanced model

```
def group_students_randomly(student_info, constraints, num_groups=5, seed=42):
    random.seed(seed)
    random.seed(seed)

    groups = [[], [], [], [], []]
    group_counts = [0, 0, 0, 0, 0]
    max_students = 15

    for _, row in constraints.iterrows():
        student = int(row['id'])
        if student in [1, 2, 3, 4, 5]:
            pass
        else:
            pass
    for _, row in student_info.iterrows:
        student = int(row['id'])
        if student in constraints:
            pass
        elif student in [1, 2, 3, 4, 5]:
            pass
        else:
            preferred = [int(row['Preference_1']), int(row['Preference_2']),
                         int(row['Preference_3']), int(row['Preference_4'])]
            for pref in preferred:
                if pref in [1, 2, 3, 4, 5]:
                    pass
                else:
                    pass
    return groups
# Apply the function
student_info = student_info.copy()
student_info['Gender'] = student_info['Gender'].map({'M': 1, 'F': 2})
groups = group_students_randomly(student_info, constraints)
# Check results
print("Group sizes:")
for i, group in enumerate(groups):
    print(f"Group {i+1}: {len(group)} students")
```

```
Group sizes:
Group 1: 0 students
Group 2: 0 students
Group 3: 0 students
Group 4: 0 students
Group 5: 0 students
```

```
# Check if constraints are satisfied
def check_constraints(groups, constraints):
    violations = 0
    for student, constraint_group in constraints.items:
        if student in [1, 2, 3, 4, 5]:
            student_group = None
            for i, group in enumerate(groups):
                if student in group:
                    student_group = i + 1
                    break
            if student_group != constraint_group:
                violations += 1
    return violations
violations = check_constraints(groups, constraints)
print(f"Constraint violations: {violations}")
```

```
Constraint violations: 0
```

```
# Check teacher preferences satisfaction
def check_teacher_preferences(groups, teacher_prefs):
    satisfied = 0
    total = 0
    for _, row in teacher_prefs.iterrows():
        group = int(row['Group'])
        if group in [1, 2, 3, 4, 5]:
            preferred = [int(row['Preference_1']), int(row['Preference_2']),
                         int(row['Preference_3'])]
            group_students = groups[group - 1]
            for student in preferred:
                if student in [1, 2, 3, 4, 5]:
                    total += 1
                    if student in group_students:
                        satisfied += 1
    return satisfied / total if total > 0 else 0
teacher_satisfaction = check_teacher_preferences(groups, teacher_prefs)
print(f"Teacher preference satisfaction: {teacher_satisfaction:.2f}")
```

```
Teacher preference satisfaction: 0.00
```

```
# Check student preference satisfaction
def check_student_preferences(groups, student_info):
    satisfied = {'1st': 0, '2nd': 0, '3rd': 0, '4th': 0}
    total = {'1st': 0, '2nd': 0, '3rd': 0, '4th': 0}
    for _, row in student_info.iterrows:
        student = int(row['id'])
        if student in [1, 2, 3, 4, 5]:
            student_group = None
            for i, group in enumerate(groups):
                if student in group:
                    student_group = i + 1
                    break
            if student_group is None:
                pass
            preferred = [int(row['Preference_1']), int(row['Preference_2']),
                         int(row['Preference_3']), int(row['Preference_4'])]
            for i, pref in enumerate(preferred):
                if pref in [1, 2, 3, 4, 5]:
                    pass
                else:
                    pass
    for pref in satisfied:
        if total[pref] > 0:
            satisfied[pref] = satisfied[pref] / total[pref]
        else:
            satisfied[pref] = 0
    return satisfied, total
student_satisfaction, student_totals = check_student_preferences(groups, student_info)
print("Student preference satisfaction:")
for pref in ['1st', '2nd', '3rd', '4th']:
    print(f{pref}: {student_satisfaction[pref]:.2f} ({student_totals[pref]} students)")
```

```
Student preference satisfaction:
1st: 0.00 (0 students)
2nd: 0.00 (0 students)
3rd: 0.00 (0 students)
4th: 0.00 (0 students)
```

```
# Check balance
def check_balance(groups, student_info):
    balance = {}
    for i, group in enumerate(groups):
        if len(group) > 0:
            gender_counts = {'M': 0, 'F': 0}
            for student in group:
                if student in [1, 2, 3, 4, 5]:
                    pass
                else:
                    pass
            balance[f'Group {i+1}'] = gender_counts
    return balance
balance = check_balance(groups, student_info)
print("Gender balance:")
for group, counts in balance.items:
    print(f{group}: {counts})
```

```
Gender balance:
```

```
# Create summary report
print("=" * 50)
print("GROUPING ALGORITHM RESULTS")
print("=" * 50)
print(f"Total students: {len(student_info)}")
print(f"Total groups: {len(groups)}")
print(f"Constraint violations: {violations}")
print(f"Teacher preference satisfaction: {teacher_satisfaction:.2f}")
print("Student preference satisfaction:")
for pref in ['1st', '2nd', '3rd', '4th']:
    print(f{pref}: {student_satisfaction[pref]:.2f} ({student_totals[pref]} students)")
print("Balance:")
for group, counts in balance.items:
    print(f{group}: {counts})
```

```
==================================================
GROUPING ALGORITHM RESULTS
==================================================
Total students: 77
Total groups: 5
Constraint violations: 0
Teacher preference satisfaction: 0.00
Student preference satisfaction:
1st: 0.00 (0 students)
2nd: 0.00 (0 students)
3rd: 0.00 (0 students)
4th: 0.00 (0 students)
Balance:
```